

MINISTRY OF EDUCATION AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

REPORT

Laboratory Work No. 4

Operating Systems

The xv6 Shell from the Outside

Student: Bianca-Andreea Tataroi
Group: FAF-242

Checked by: Reițman Patricia

Chișinău – 2026

Contents

1 Objective **2**

2 Part 1 – The sleep Program **2**
2.1 Observation 3

3 Part 2 – Comparison Between Bash and xv6 **3**
3.1 Redirection 5
3.2 Pipes 6
3.3 Grouped Commands 6
3.4 Command Sequences 6
3.5 Background Execution 6
3.6 Wildcard Expansion 7
3.7 Environment Variable Expansion 7
3.8 Quotation Marks 7
3.9 Executing Programs After Changing Directory 8
3.10 History and Completion 10

4 Part 3 – Command-Line Arguments **10**
4.1 Testing in Bash 11
4.2 Testing in xv6 12
4.3 Observation 13

5 Part 4 – Reading the xv6 Shell Source Code **13**
5.1 Command Types 13
5.2 The cd Command 14
5.3 Wildcard and Variable Expansion 14

6 GitHub Repository **15**

7 Conclusion **15**

1 Objective

The objective of this laboratory work was to study the behaviour of the xv6 shell and compare it with the Bash shell available in Linux.

During the laboratory work, several shell features were tested, including command execution, redirection, pipes, command sequences, background processes, wildcard expansion, variable expansion and quotation marks.

A small program for inspecting command-line arguments was also created. Finally, the source code of the xv6 shell was examined in order to understand how commands are parsed and executed.

2 Part 1 – The sleep Program

The first task was to inspect the xv6 user interface in order to find the function responsible for pausing a process.

The following command was used:

```
cd ~/xv6-riscv
grep -n "pause\|sleep" user/user.h
```

The output was:

```
25:int pause(int);
```

This showed that in the xv6 version used in this laboratory, the `pause()` function is available.

A new program called `sleep.c` was created in the `user/` directory.

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    if(argc != 2){
        fprintf(2, "usage: sleep <ticks>\n");
        exit(1);
    }

    pause(atoi(argv[1]));
    exit(0);
}
```

The program checks whether exactly one argument is provided. The argument is converted to an integer using `atoi()` and is then passed to `pause()`.

The following entry was added to the `UPROGS` section of the Makefile:

```
$U/_sleep\
```

After rebuilding xv6, the program was tested using:

```
sleep 20
sleep
```

When `sleep 20` was executed, the process paused for the specified number of ticks.

When the program was executed without an argument, the following message was displayed:

```
usage: sleep <ticks>
```



Figure 1: Testing the sleep utility in xv6

2.1 Observation

This task demonstrated how a user-space xv6 program can use an existing operating system function.

It also showed the importance of validating command-line arguments before calling a system service.

3 Part 2 – Comparison Between Bash and xv6

The second task was to compare the Bash shell in Linux with the xv6 shell.

First, a test directory was created in Ubuntu:

```
mkdir -p ~/shtest
cd ~/shtest
touch README
```

The following commands were then executed:

```
echo hello > f ; cat < f
ls | grep READ
(echo a; echo b) | wc
```

```
echo one ; echo two
echo bg &
ls *
echo $HOME
echo "a b"
mkdir -p d ; cd d ; ls ..
cd ~/shtest
```

The results in Bash are shown below.



Figure 2: Testing shell features in Bash

The same commands were then executed in the xv6 shell.

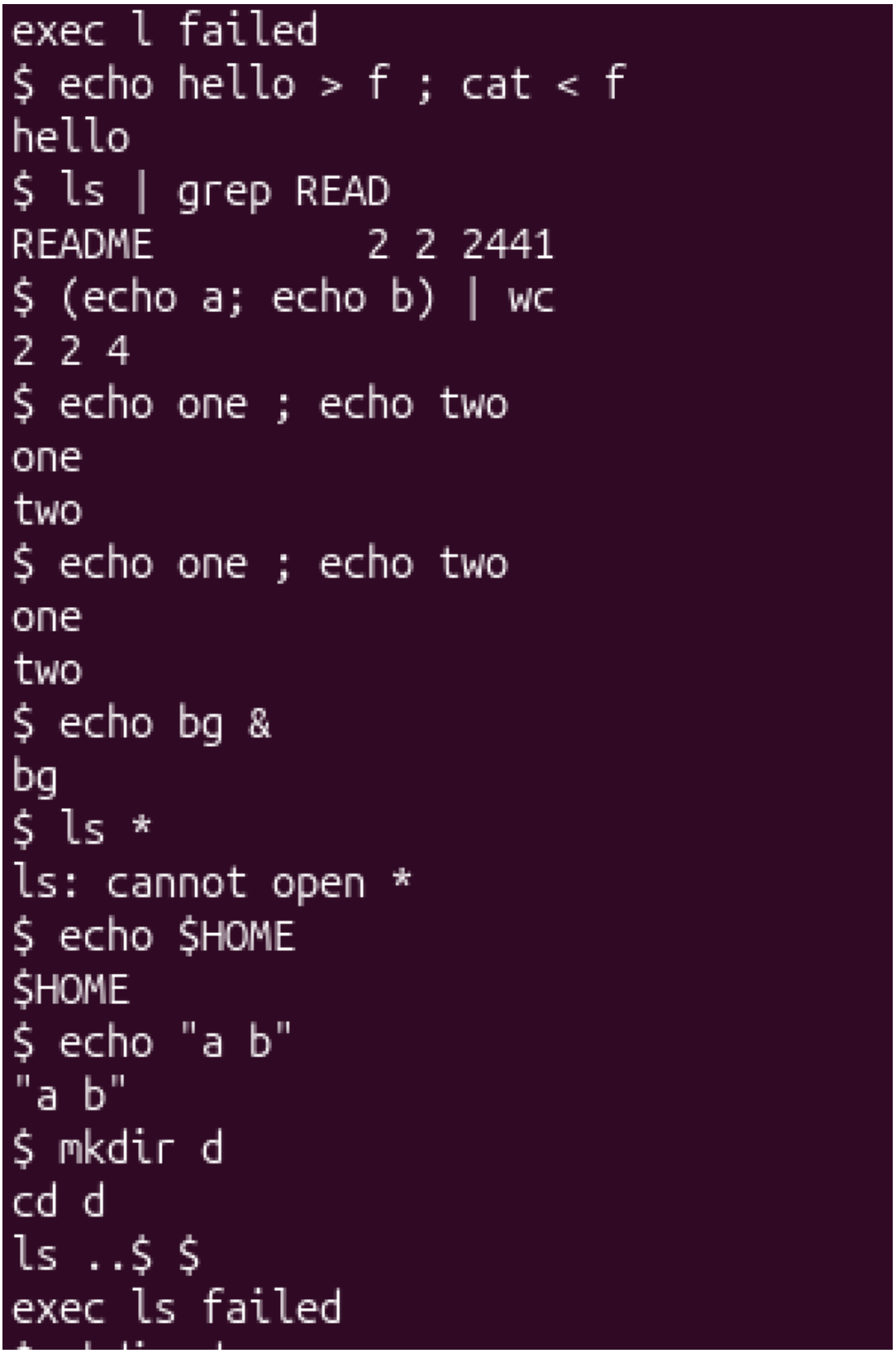


Figure 3: Testing the same shell features in xv6

3.1 Redirection

The command:

```
echo hello > f ; cat < f
```

worked in both Bash and xv6.
The output was:

```
hello
```

This demonstrates that xv6 supports input and output redirection.

3.2 Pipes

The command:

```
ls | grep READ
```

also worked in both shells.
In xv6, the output included:

```
README
```

This shows that the xv6 shell supports pipelines.

3.3 Grouped Commands

The command:

```
(echo a; echo b) | wc
```

produced:

```
2 2 4
```

Therefore, the xv6 shell is able to execute grouped commands and send their output through a pipe.

3.4 Command Sequences

The command:

```
echo one ; echo two
```

produced:

```
one
two
```

This shows that the semicolon can be used to execute multiple commands sequentially in xv6.

3.5 Background Execution

The command:

```
echo bg &
```

worked in xv6 and printed:

```
bg
```

This demonstrates that the `&` operator for background execution is supported.

3.6 Wildcard Expansion

In Bash:

```
ls *
```

expanded the wildcard and displayed the matching files.
In xv6, the command produced:

```
ls: cannot open *
```

This means that the xv6 shell does not expand the `*` wildcard.
Instead, the string `*` is passed directly to the `ls` program.

3.7 Environment Variable Expansion

In Bash:

```
echo $HOME
```

produced:

```
/home/bianca
```

In xv6, the result was:

```
$HOME
```

This shows that the xv6 shell does not implement environment variable expansion.

3.8 Quotation Marks

In Bash:

```
echo "a b"
```

produced:

```
a b
```

The quotation marks were interpreted and removed by Bash.
In xv6, the result was:

```
"a b"
```

Therefore, the xv6 shell does not process quotation marks in the same way as Bash.

3.9 Executing Programs After Changing Directory

Another important difference was found after changing directories.

The following commands were used:

```
mkdir d
cd d
ls
```

Inside the new directory, the command:

```
ls
```

failed with:

```
exec ls failed
```

However, executing:

```
../ls ..
```

worked successfully.

```
$ mkdir d
exec ls failed
$ cd d
cannot cd d
$ ls
exec ls failed
$ ../ls ..
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
forktest       2 5 18136
grep           2 6 44184
init           2 7 35976
kill           2 8 35512
ln             2 9 35304
ls             2 10 42880
mkdir          2 11 35568
rm             2 12 35552
sh             2 13 58312
stressfs       2 14 36416
usertests      2 15 206800
grind          2 16 51824
wc             2 17 37728
zombie         2 18 34848
logstress      2 19 37560
forphan        2 20 36352
dorphan        2 21 35800
sync           2 22 34944
sleep          2 23 35280
```

The reason is that xv6 does not search for programs using a Bash-style `PATH` environment variable.

After changing directory, the executable must be referenced using its relative or absolute path.

3.10 History and Completion

Bash supports interactive features such as:

- command history using the Up Arrow;
- tab completion using Tab;
- reverse history search using Ctrl+R.

The basic xv6 shell does not provide these features.

For everyday use, command history and tab completion would be among the most useful missing features because they reduce the amount of typing required.

4 Part 3 – Command-Line Arguments

The third task was to determine which component is responsible for processing wildcards, variables and quotation marks.

An xv6 program called `args.c` was created.

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    printf("argc = %d\n", argc);

    for(int i = 0; i < argc; i++)
        printf("argv[%d] = [%s]\n", i, argv[i]);

    exit(0);
}
```

The following entry was added to the Makefile:

```
$U/_args\
```

A Linux version of the program was also created:

```
#include <stdio.h>

int
main(int argc, char *argv[])
{
    printf("argc = %d\n", argc);
```

```
    for(int i = 0; i < argc; i++)
        printf("argv[%d] = [%s]\n", i, argv[i]);

    return 0;
}
```

It was compiled with:

```
gcc -o args args_linux.c
```

4.1 Testing in Bash

The following command was executed:

```
./args one two three
```

The output was:

```
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
```

The value of `argc` is 4 because the program name itself is stored in `argv[0]`.

The next test was:

```
./args *.c "x y"
```

The output was:

```
argc = 3
argv[0] = [./args]
argv[1] = [args_linux.c]
argv[2] = [x y]
```

This shows that Bash expanded `*.c` into the matching filename before the program was executed.

It also interpreted the quotation marks and passed `x y` as one argument.

The following test was also performed:

```
./args $HOME
```

The output was:

```
argc = 2
argv[0] = [./args]
argv[1] = [/home/bianca]
```

This confirms that Bash expanded the `$HOME` variable.

The quoted string test was:

```
./args 'single quoted'
```

The output was:

```
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
```

The quotation marks were removed by Bash and the text was passed as a single argument.

4.2 Testing in xv6

The same tests were then performed inside xv6.

The first test:

```
args one two three
```

produced:

```
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
```

The second test:

```
args *.c "x y"
```

produced:

```
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
```

This is different from Bash.

The wildcard `*.c` was passed directly as text, and the quoted string was split into two arguments.

The variable test:

```
args $HOME
```

produced:

```
argc = 2
argv[0] = [args]
argv[1] = [$HOME]
```

Therefore, xv6 did not expand the variable.

Finally:

```
args 'single quoted'
```

produced:

```
argc = 3
argv[0] = [args]
argv[1] = ['single]
argv[2] = [quoted']
```

This shows that single quotation marks are also not interpreted by the xv6 shell.

4.3 Observation

The results prove that wildcard expansion, environment-variable expansion and quotation processing are performed by the shell before the program starts.

The program itself simply receives the already prepared `argc` and `argv` values.

Bash implements these advanced transformations, while the xv6 shell does not.

Multiple spaces used between ordinary arguments are treated as separators and are not preserved inside `argv`.

5 Part 4 – Reading the xv6 Shell Source Code

The final task was to inspect the source code of the xv6 shell.

The following commands were executed:

```
wc -l user/sh.c
grep -n "#define" user/sh.c
grep -n "case " user/sh.c
grep -n "cd" user/sh.c | head
```

The shell source file contained:

```
499 user/sh.c
```

The main command types were defined as:

```
#define EXEC 1
#define REDIR 2
#define PIPE 3
#define LIST 4
#define BACK 5
#define MAXARGS 10
```

5.1 Command Types

The five command types have the following purposes:

- `EXEC` – execution of an ordinary command;
- `REDIR` – input or output redirection;
- `PIPE` – connecting commands using a pipe;
- `LIST` – sequential commands separated by a semicolon;
- `BACK` – execution of a command in the background.

The shell parser recognizes several special symbols:

```
< | > & ; ( )
```

The meaning of some of these symbols is:

- `<` – input redirection;
- `>` – output redirection;
- `|` – pipeline;
- `;` – command sequence;
- `&` – background execution;
- `( )` – grouped commands.

5.2 The cd Command

The search for `cd` showed that it is handled specially inside the main shell loop.

The relevant code is:

```
if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
    // Chdir must be called by the parent, not the child.
    cmd[strlen(cmd) - 1] = 0;

    if (chdir(cmd + 3) < 0)
        fprintf(2, "cannot cd %s\n", cmd + 3);
}
```

The `cd` command cannot be executed like an ordinary external program because changing the directory in a child process would not change the directory of the parent shell.

Therefore, `chdir()` must be called directly by the shell process.

5.3 Wildcard and Variable Expansion

The xv6 shell parser contains the following definitions:

```
char whitespace[] = " \t\r\n\v";
char symbols[] = "<|>&;()";
```

The shell only recognizes a small set of special characters.

There is no Bash-style handling of wildcard expansion or environment variables.

This explains the results obtained earlier:

```
ls *
```

passed the `*` character directly to the program, while:

```
echo $HOME
```

printed `$HOME` instead of expanding it.

6 GitHub Repository

The source files and the final report for Laboratory Work No. 4 were uploaded to GitHub.

The repository contains the files used in this laboratory work, including:

- `sleep.c`;
- `args.c`;
- `args_linux.c`;
- `Makefile`;
- the final laboratory report.

The repository can be accessed at:

`https://github.com/Biancatataroi/Operating-Systems-Lab4`

7 Conclusion

In this laboratory work, the xv6 shell was examined and compared with the Bash shell used in Linux.

The first part demonstrated how a user-space xv6 program can use an existing operating system service. The `sleep` program checked its argument, converted it to an integer and used the `pause()` function to suspend execution.

The second part demonstrated that xv6 supports several fundamental Unix shell mechanisms, including redirection, pipes, sequential commands, grouped commands and background execution.

At the same time, important differences between Bash and xv6 were observed. The xv6 shell does not perform wildcard expansion, environment-variable expansion or Bash-style quotation processing. It also does not provide advanced interactive features such as command history and tab completion.

The command-line argument experiments showed that the shell is responsible for interpreting and transforming command lines before the requested program is executed. Bash performs wildcard expansion, variable expansion and quotation processing, while xv6 passes most of these characters directly to programs.

Finally, the analysis of `user/sh.c` showed how the xv6 shell represents commands internally using five main command types: `EXEC`, `REDIR`, `PIPE`, `LIST` and `BACK`. The source code also explained why the `cd` command must be handled directly by the shell process.

Overall, this laboratory work provided a practical understanding of the responsibilities of a command-line shell and demonstrated how the simple xv6 shell implements the essential principles of Unix command execution.